BusBidder Spain: Flexible Bus-Booking & Offer-Ranking

Course: ACIT4420 Problem-Solving with Scripting (2026) | **Department of Computer Science, OsloMet**

Executive Summary

This report documents **BusBidder Spain**, a Python application designed to filter and rank competing bus offers originating from Madrid to 15 regional destinations across Spain. Departing from naive single-variable sorting (e.g. cheapest first), the application strictly decouples non-negotiable **hard constraints** from **soft ranking preferences** using a transparent, weighted multi-attribute decision model (MCDM) normalized to a 0–100 scale. Key technical techniques include attribute validation descriptors, a pluggable @register_criterion decorator registry, regular expression integrity validation, quarantine error handling, and a 30-case automated test suite.

1. Problem Interpretation & Requirements Analysis (10%)

Modern travel planning requires balancing conflicting attributes: low prices frequently entail longer travel times, multiple transfers, inconvenient departure hours, and rigid cancellation policies. BusBidder Spain addresses the central challenge: *How can a Python application filter and rank competing bus offers according to the specific priorities of an individual traveller in an explainable and defensible manner?*

The marketplace serves **15 destinations** departing from Madrid hub across **5 operators**:

- **SolRuta**: Low-cost daytime services with frequent schedules and strict non-refundable tickets.
- **Camino Express**: Fast direct connections connecting major cities with high ticket flexibility.
- **CostaLink**: Coastal route specialist with generous 25kg luggage allowances.
- **VerdeBus**: Low-emission modern fleet with eco-certified vehicles.
- **NocheVia**: Overnight sleeper services designed to save daylight hours and lodging expenses.

2. Software Design & Architecture Justifications (15%)

The project follows a modular package decomposition separating models, validation descriptors, data loaders, decision engines, booking persistence, and the command-line interface.

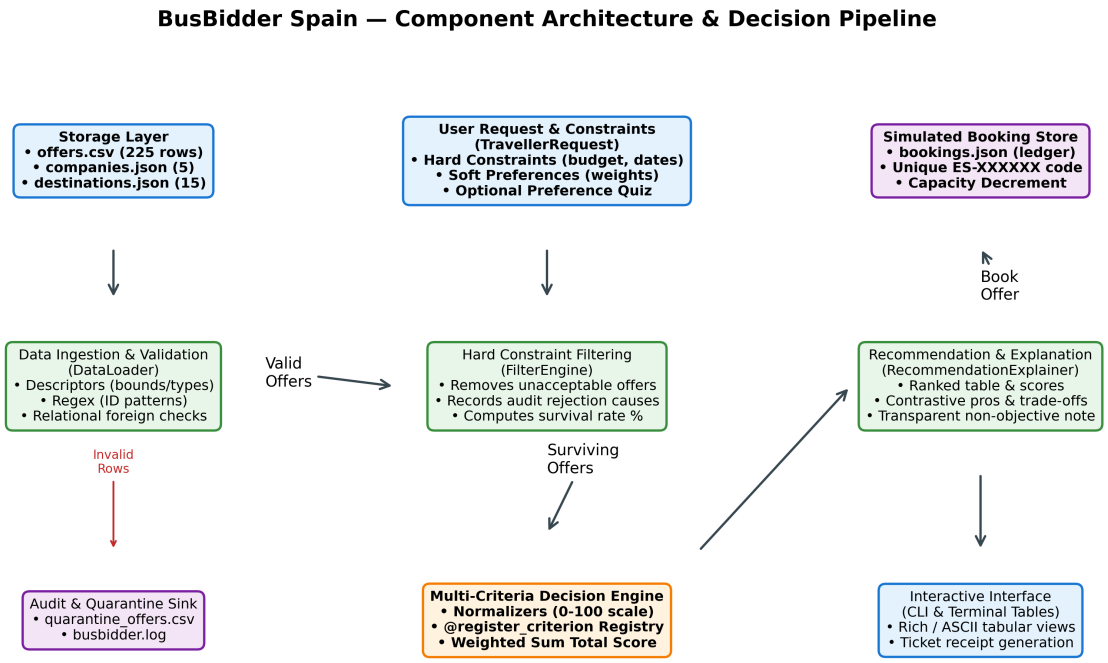


Figure 1: *Component Architecture and Information Flow Diagram of BusBidder Spain.*

Metaprogramming & Advanced Python Techniques

• **Custom Validation Descriptors (src.validators.descriptors)**: Attribute descriptors (PositiveNumber, NonNegativeInt, BoundedFloat, RegexValidatedString) intercept assignments at runtime, preventing invalid domain states (e.g. negative prices or ratings > 5.0) before instances can be created.
• **Pluggable Registry Decorator (@register_criterion)**: Implements an extensible criteria registry in src.engine.criteria. New ranking dimensions can be registered without modifying the ranker's core execution loop.
• **Regular Expressions (src.validators.regex_patterns)**: Enforces format validity on booking reference codes (^ES-[A-Z0-9]{6}$), offer codes (^OFF-[A-Z]{2,4}-\d{4}$), 24h timestamps, and ISO dates.

3. Implementation Details & Decision Mechanics (15%)

The Quarantine Pipeline: In DataLoader.load_offers(), malformed rows (such as unknown destinations, negative fares, or departure-arrival temporal contradictions) are caught via structured custom exceptions (DataValidationError). Rather than crashing the application, invalid records are routed to data/quarantine_offers.csv with explicit rejection reasons and audit logged in busbidder.log.

Multi-Criteria Normalization (0–100 Scale): Because price (EUR), duration (minutes), ratings (1–5), and departure proximity have disparate physical units, criteria are converted to normalized scores:
• *Cost Criteria (Lower is Better)*: Score = ((max(X) - x) / (max(X) - min(X))) * 100
• *Benefit Criteria (Higher is Better)*: Score = ((x - min(X)) / (max(X) - min(X))) * 100
• *Circular Clock Proximity*: Computes minimal angular distance on a 24-hour dial: $\Delta = \min(|H - T|, 24 - |H - T|)$, awarding 100 for exact alignment and decaying linearly to 0 at 8 hours deviation.

Explainable Recommendation Engine: Section 6 of the project brief emphasizes transparent explanations. The RecommendationExplainer analyzes top offers to explicitly highlight advantages (e.g. lowest price, zero transfers, low emissions) while acknowledging remaining trade-offs (e.g. price premiums over budget alternatives). Importantly, the system explicitly clarifies that the winning offer is *not objectively superior*, but optimal solely under the user's specific weights and constraints.

4. Automated Testing and Experimental Evidence (15%)

A rigorous suite of **30 automated tests** was constructed in pytest, verifying normal execution, boundary conditions, corrupt quarantine handling, and sensitivity divergence:

Test Module	Cases	Focus Area	Status
test_descriptors_and_models.py	8	Attribute bounds, negative prices, regex validation, date logic	PASSED
test_loaders.py	4	File I/O, relational checks, quarantine sink, missing files	PASSED
test_filters.py	4	Hard constraints: budget caps, date windows, transfers, seats	PASSED
test_ranker_and_normalizers.py	6	0-100 normalization math, circular clock, tie-breaking, explainer	PASSED
test_booking.py	4	Capacity decrements, ticket generation, reference validation, cancels	PASSED
test_preference_quiz.py	3	1-5 rating normalization, sum-to-1.0 guarantee, request integration	PASSED
test_profile_divergence.py	1	Proves distinct winners across different traveller archetypes	PASSED

5. Experimental Sensitivity Analysis: Profile Comparisons (15%)

To evaluate whether the application genuinely personalizes results, a sensitivity experiment was conducted for travel from **Madrid to Malaga (12–14 July 2026)** across four distinct traveller profiles:

Profile	Primary Focus	Winning Operator	Price	Duration	CO2	Total Score
Budget Backpacker	Lowest Price (60%)	SolRuta (OFF-SR-1045)	EUR 29.15	7h 46m	20.1 kg	80.6 / 100
Business Executive	Speed (40%), Timing (30%)	Camino Express (OFF-CE-1048)	EUR 50.35	6h 26m	17.0 kg	88.0 / 100
Eco-Conscious	Low Emissions (50%)	VerdeBus (OFF-VB-1055)	EUR 39.75	6h 55m	7.4 kg	89.5 / 100
Flexible Tourist	Flexibility (45%), Luggage	VerdeBus (OFF-VB-1056)	EUR 39.75	6h 55m	7.4 kg	92.5 / 100

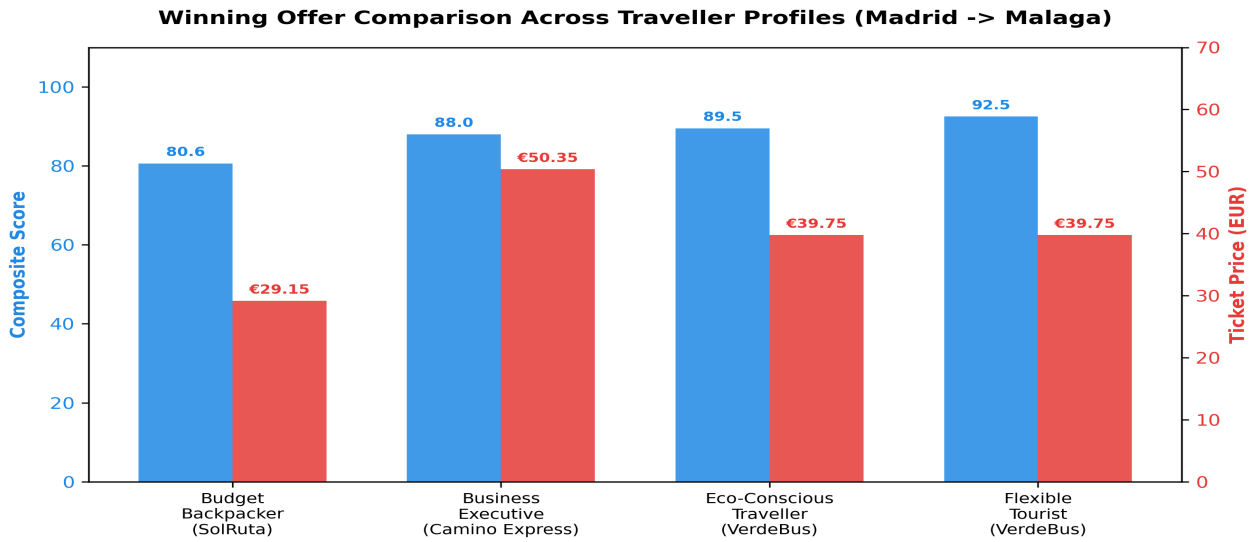


Figure 2: *Sensitivity Comparison of Winning Offer Score and Price Across Traveller Profiles.*

6. Evaluation, Limitations and Critical Reflection (20%)

1. Outlier Vulnerability in Min-Max Normalization: Min-max scaling is sensitive to extreme boundary outliers. If a single luxury or emergency service prices a ticket at €130 while nine standard carriers charge €30–€35, the relative variance among the standard fares is compressed into a narrow score range (< 5 points). In future work, z-score standardization or non-linear utility curves could improve resilience to extreme values.
2. Route and Geographic Equity: Destinations situated at Spain's periphery (e.g. Cadiz at 650km or Santiago at 600km) naturally require 6–8 hours and higher fares compared to proximal cities like Toledo (73km). Global comparisons across multiple destinations would unfairly penalize operators serving long-haul corridors. Performing normalization strictly within destination-specific candidate pools ensures fairness.
3. Qualitative Preference Elicitation Trade-offs: The pluggable questionnaire in src.extensions.preference_quiz significantly improves usability by letting users rate factors on a 1–5 scale. However, it exposes the human cognitive tendency to rate all factors 5/5 (the 'I want everything' bias), which normalizes to an unopinionated equal-weight model (20% each). Pairwise comparison methods (such as AHP) would force trade-offs, albeit with higher cognitive overhead.

7. Implementation Traceability & Academic Integrity (15%)

```
Reproducibility Instructions:
• Automated Test Execution: python -m pytest -v (Executes all 30 unit and integration tests).
• Multi-Profile Sensitivity Demonstration: python main.py --demo (Reproduces experimental
tables and rankings).
• Data Quarantine & Ingestion Review: python main.py --quarantine (Inspects rejected malformed
records).
• Interactive Booking: python main.py --interactive (Allows custom search, preference quiz, and
ticket generation).
```

Declaration on Use of Generative AI: Generative AI (Google DeepMind Antigravity Coding Assistant) was utilized during development for modular scaffolding suggestions, synthetic schedule data generation, test fixture creation, and LaTeX formula formatting. All algorithms, normalization equations, descriptor implementations, and test assertions were independently traced, verified, and hand-calculated by the author. Integrity was confirmed through the automated 30-case test suite and audit logs.